STAND-UP

Reflect with ORID



© 2026 Inspire | Confidential

REVIEW

Share Your API Homework



© 2026 Inspire | Confidential

Connect your contract, code and tests

Take equal turns sharing your graded work.

1. Show one success or error example in your OpenAPI, code and tests.
2. Give each person specific feedback. Keep the submission and score unchanged.

Produce

Peer comments and questions.

20 minutes · whole activity

© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

SQL Tables and Statements



© 2026 Inspire | Confidential

One quote, one row

SQL · Structured Query Language

Quote · Java object

`id`	**17**
`weight_kg`	**3**
`zone`	**local**
`service`	**standard**
`amount_cents`	**1300**

quotes · Table

id	weight_kg	zone	service	amount_cents
17	3	local	standard	1300

One quote per row

One field per column

Supplied example · Amount in cents

A table contains rows. Each column stores one field.

© 2026 Inspire | Confidential

Types and constraints describe a column

Column	SQL type	Meaning
`id`	BIGINT · PRIMARY KEY	Identifies one row
`weight_kg`	INTEGER · NOT NULL	Whole kilograms, required
`zone, service`	VARCHAR · NOT NULL	Text, required
`amount_cents`	INTEGER · NOT NULL	Whole cents, required

> NOT NULL rejects a missing value. Business rules still check valid weights and services.

© 2026 Inspire | Confidential

DDL defines structure; DML works with rows

Structure · DDL

CREATE TABLE

Define the table, column types and constraints.

Data · DML

INSERT · SELECT
UPDATE · DELETE

Add, read, change or remove records.

id	weight_kg	zone	service	amount_cents
17	3	local	standard	1300

DDL: Data Definition Language. DML: Data Manipulation Language.

© 2026 Inspire | Confidential

Create the quotes table

```
CREATE TABLE quotes (
  id BIGINT PRIMARY KEY,
  weight_kg INTEGER NOT NULL,
  zone VARCHAR(20) NOT NULL,
  service VARCHAR(20) NOT NULL,
  amount_cents INTEGER NOT NULL
);
```

The SQL practice uses explicit ids. PRIMARY KEY identifies each row; NOT NULL requires a value.

© 2026 Inspire | Confidential

Four operations on the same quote

SQL examples · Illustrative changes, not application results

INSERT

Add row 17

```
INSERT INTO quotes (id, weight_kg, zone, service, amount_cents)
VALUES (17, 3, 'local', 'standard', 1300);
```

SELECT

Read: standard

```
SELECT id, service FROM quotes WHERE id = 17;
```

UPDATE

20 kg · express · 7900 cents

```
UPDATE quotes SET weight_kg = 20, zone = 'local',
service = 'express', amount_cents = 7900 WHERE id = 17;
```

DELETE

Remove row 17

```
DELETE FROM quotes WHERE id = 17;
```

WHERE selects the target rows. Check the condition before changing data.

© 2026 Inspire | Confidential

Which statement defines the table?

CREATE TABLE

INSERT · SELECT · UPDATE · DELETE

Which statements work with records?

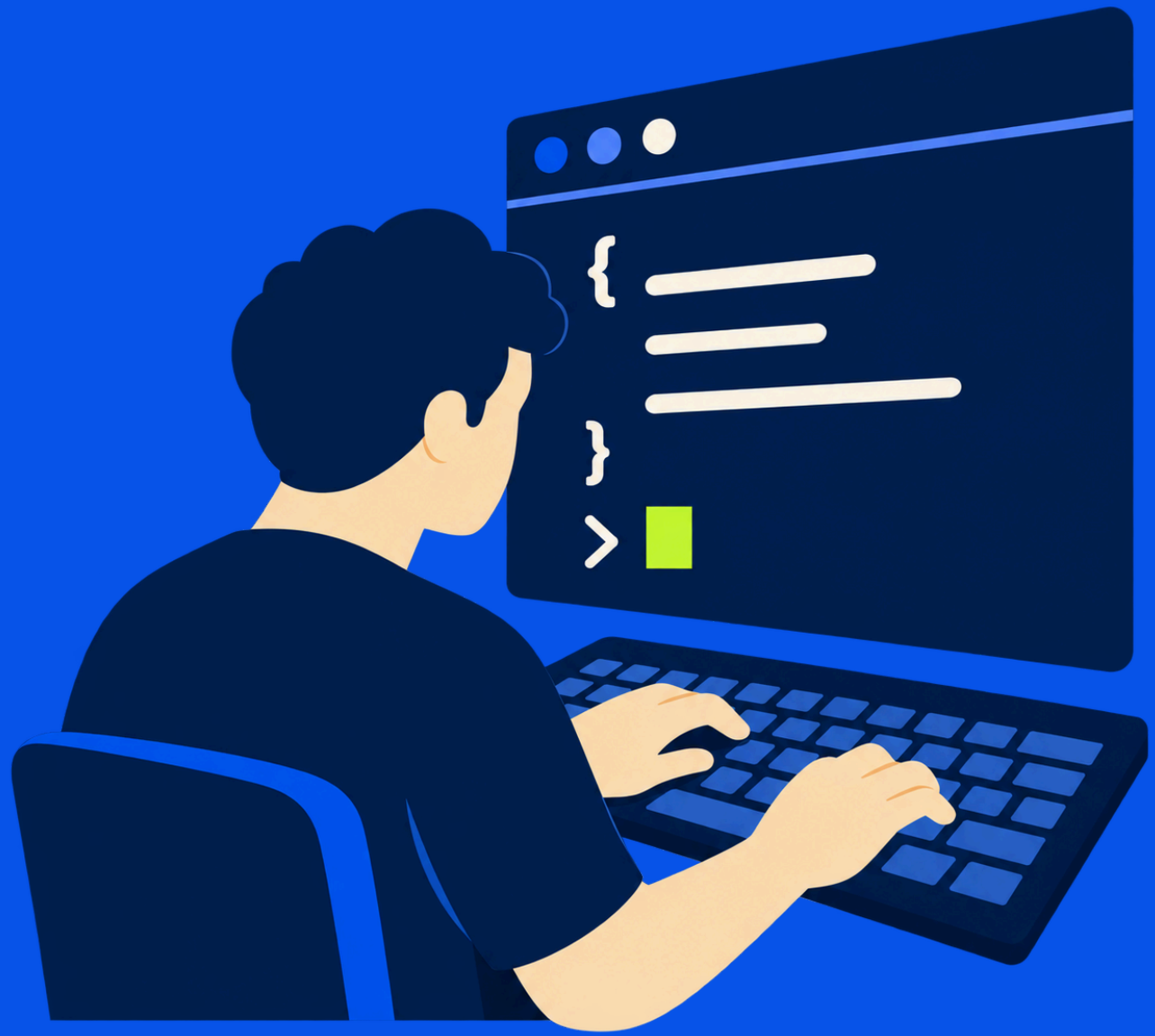
Next: follow the instructor in the separate SQL practice database.

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Create and Query the Quotes Table



© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

Database Migrations with Flyway



© 2026 Inspire | Confidential

A new database needs the same table

Your SQL practice

quotes exists

id · weight_kg · zone

service · amount_cents

You created the table by hand.

Another machine or test database

Empty database

No quotes table

The project needs a repeatable way to create the same structure.

© 2026 Inspire | Confidential

Files in Git, migrations tracked by Flyway





Versioned files

Share SQL files with the project.

Migration history

Apply pending versions in order.

Record what has run in each database.

Each environment has its own migration history.

© 2026 Inspire | Confidential

Read the migration file name

src/main/resources/db/migration/

V1	__	create_quotes	.sql
Version	Two underscores	Description	SQL file

```
CREATE TABLE quotes ( ... );
```

Structure preview · Full column definitions are written in the next Demo.

V1 creates the quotes table. The migration file travels with the application.

© 2026 Inspire | Confidential

The history shows which versions have run

01

Empty database

V1 is pending.

02

Apply V1

Create quotes and record the migration.

03

Start again

V1 is recorded.
No pending version.

`flyway_schema_history` Version 1 · create quotes · success

Expected sequence · No migration has been executed by this slide.

Keep applied scripts unchanged. Add a new version for a later structure change.

© 2026 Inspire | Confidential

Create the schema, then check consistency

Flyway migrations	Create the schema from versioned SQL.
Flyway validation	Check migration files against migration history.

What does V1 contain?

How does Flyway find the next migration?

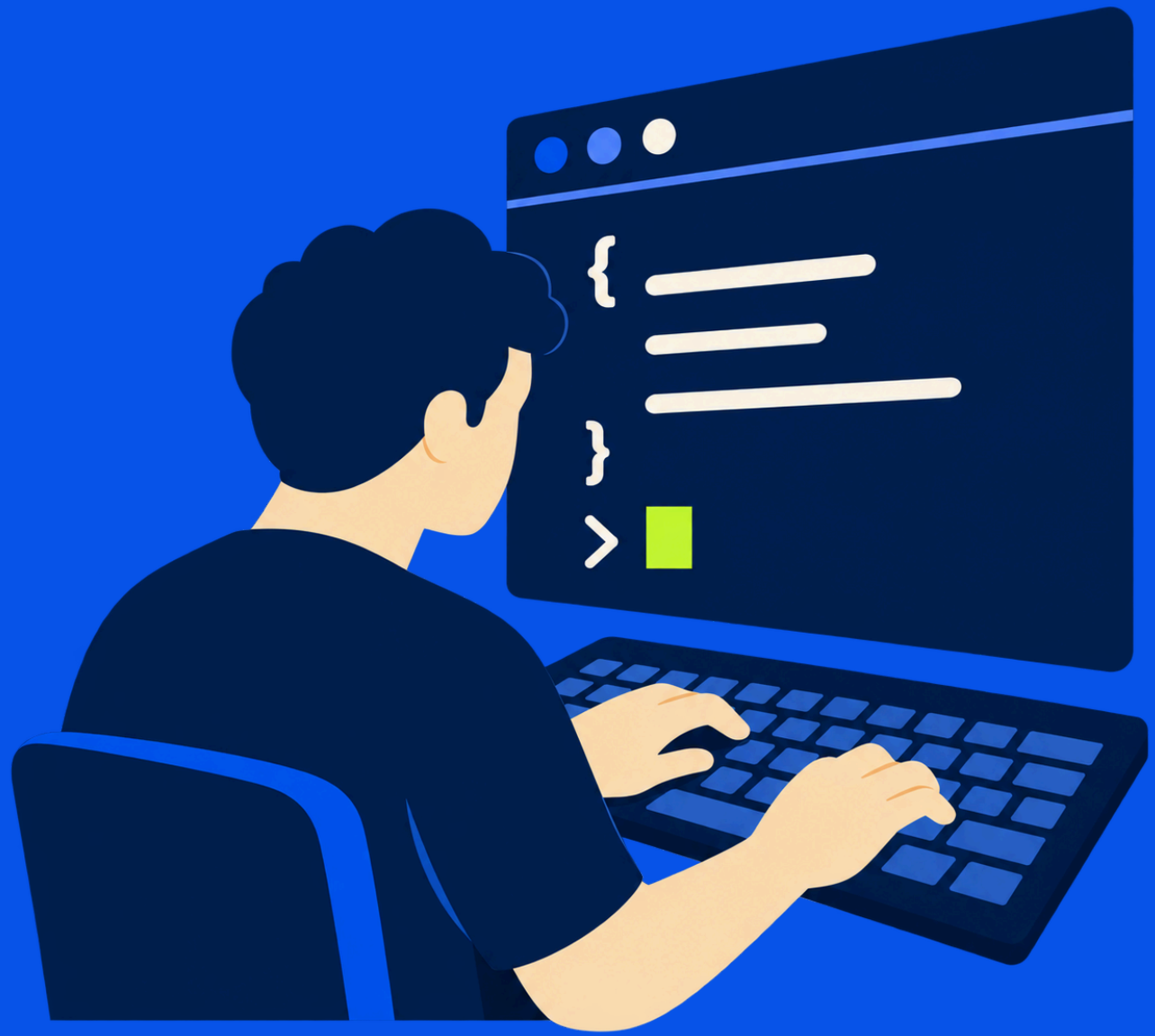
Next: write V1, start the application and inspect the database.

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Initialize the Database with Flyway V1

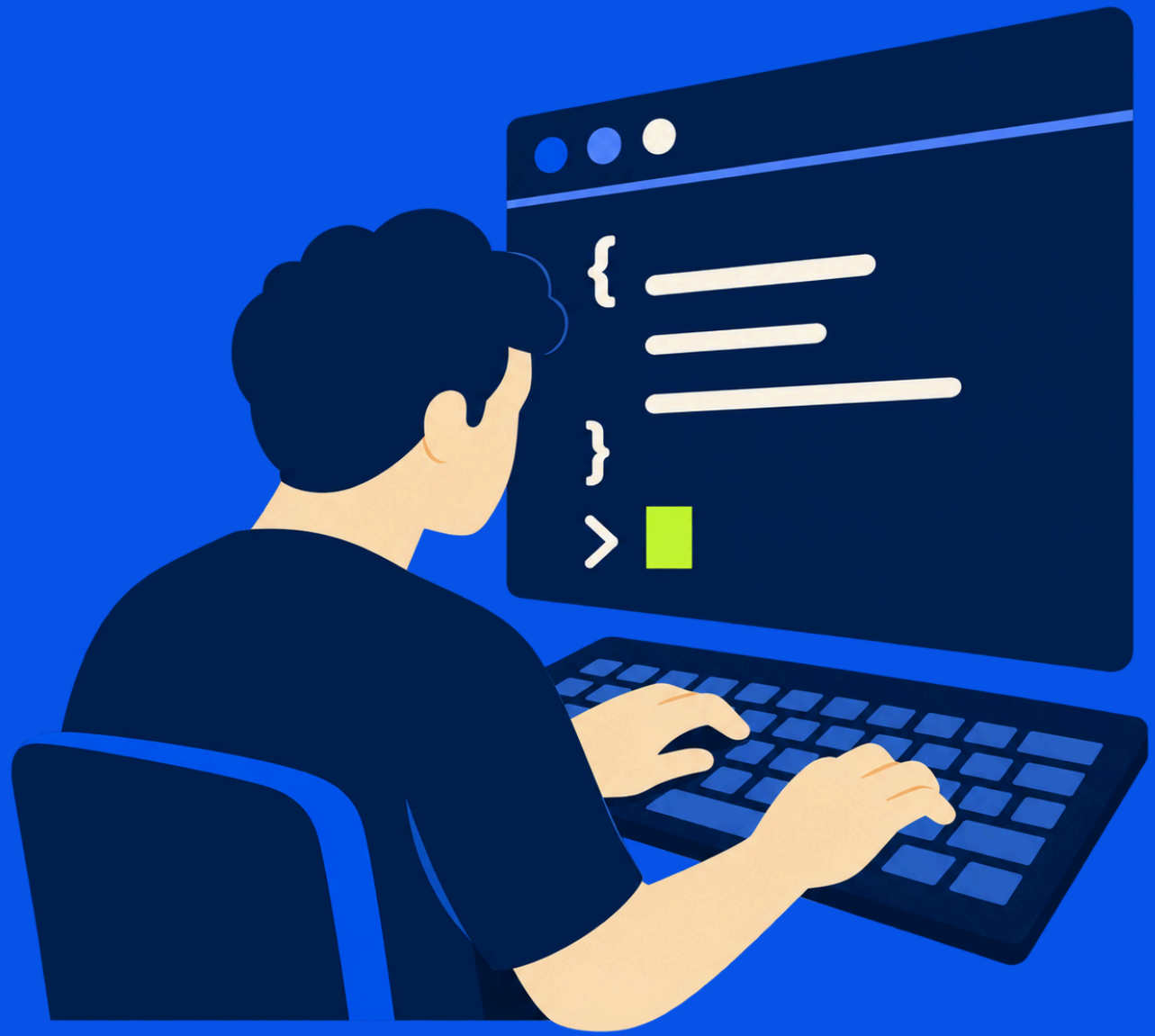


© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Test the POST Database Write



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Write Your POST Storage Check



© 2026 Inspire | Confidential

Query the id returned by your POST

Adapt the demonstrated storage test.

1. Read the generated id from the response.
2. Use that id to check the SQL row count and all stored fields.
3. Run the test. Explain its actual failure before adding persistence.

Produce

Your test, actual result and explanation.

25 minutes · whole activity

© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

ORM, JPA and Spring Data JPA



© 2026 Inspire | Confidential

A successful response still needs a storage check

POST /quotes

201 Created

Read the generated id from the response.

SQL check · Same id

Expected: 1 row

With the memory repository:

Expected with memory storage: 0 rows

Expected starting failure · Compare with your actual test result.

> The API returned a quote. The database write is still missing.

© 2026 Inspire | Confidential

ORM maps object state to table data

ORM · Object-Relational Mapping

Quote · Java object

`id`	**17**
`weight_kg`	**3**
`zone`	**local**
`service`	**standard**
`amount_cents`	**1300**

quotes · Table

id	weight_kg	zone	service	amount_cents
17	3	local	standard	1300

One quote per row

One field per column

Supplied example · Amount in cents

JPA uses an entity mapping to persist object state. Quote remains the existing API result.

© 2026 Inspire | Confidential

Three names, three roles

Specification	**JPA**	Defines persistence APIs and mapping annotations.
Provider	**Hibernate**	Implements JPA and performs persistence work.
Repository support	**Spring Data JPA**	Provides repository implementations using JPA.

Spring Data JPA uses a JPA provider such as Hibernate.

© 2026 Inspire | Confidential

Connect JPA to the existing repository

Controller	calls	**QuoteService**
QuoteService	uses	**QuoteRepository**
QuoteRepository	extends	**JpaRepository<Quote, Long>**

> Spring Data provides the repository implementation. Hibernate is the JPA provider.

The Service uses QuoteRepository directly. Quote is the entity and remains the current API response.

© 2026 Inspire | Confidential

Map the entity to the existing table

Existing request DTO

ParcelInput

weight_kg · zone · service

The Service calculates the amount.

Existing class, now mapped

Quote

Entity / Table

quotes

Id / GeneratedValue

id · database generated

Fields

weight_kg · zone · service
amount_cents

Created by V1

quotes

id · weight_kg
zone · service
amount_cents

Flyway checks migration history. Hibernate validate checks entity mappings against this table.

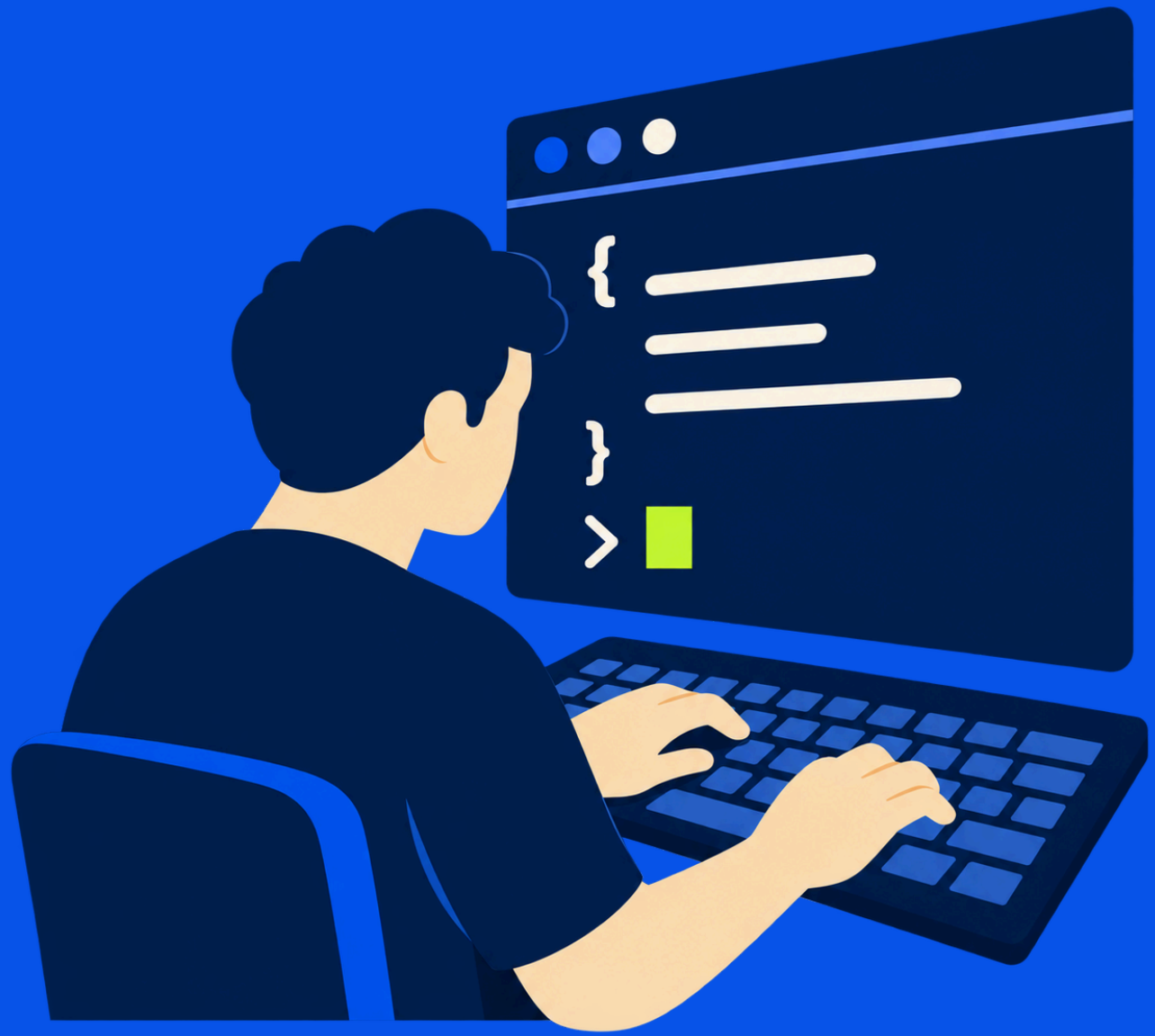
Next: add annotations to Quote and switch to QuoteRepository. Rerun the same storage test and existing API tests.

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Map QuoteEntity and Save the Quote



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Implement Storage and Check an Application Restart



© 2026 Inspire | Confidential

Keep the same quote across writes and reads

Continue from your own storage test.

1. Complete the entity mapping and database repository.
2. Use the existing reset and seed helpers. Run the storage and API tests.
3. Follow the file-database restart check. Use the same id and file path.

Produce

Code and test results; SQL and GET evidence after a restart.

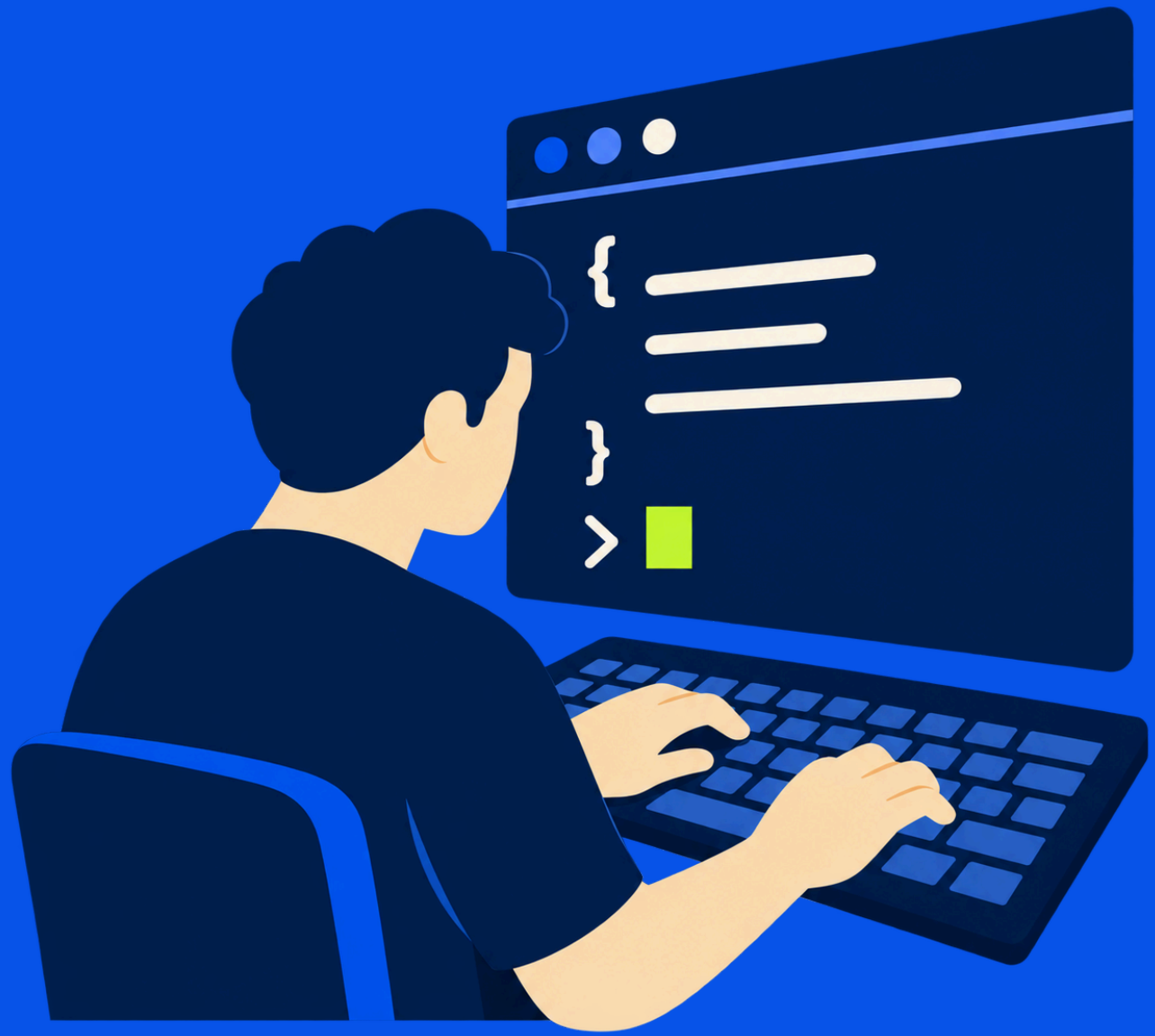
30 minutes · whole activity

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Drive a Batch Update with One Test



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Implement Batch Updates and Rollback with TDD



© 2026 Inspire | Confidential

Check the final rows after a failed batch

Keep one API and database test for the batch rule.

1. Implement the batch service change and run the test.
2. Expect HTTP 404 for a missing quote, then re-read the saved quote through the repository.
3. Place the transaction around the full batch. Rerun the same test and regression checks.

Produce

The batch endpoint, one rollback test and your explanation.

35 minutes · whole activity

© 2026 Inspire | Confidential

REVIEW

Explain the Batch Transaction Boundary



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Show Storage, Migration and Restart Evidence



© 2026 Inspire | Confidential

Connect each claim to a result

Use evidence from your own project.

1. Connect the POST response id to the SQL row and mapping.
2. Show V1 history and the same quote after a file-database restart.
3. Use the batch test to explain rollback. Correct one issue and rerun the relevant check.

Produce

One correction, its result and any remaining problem.

35 minutes · whole activity

© 2026 Inspire | Confidential

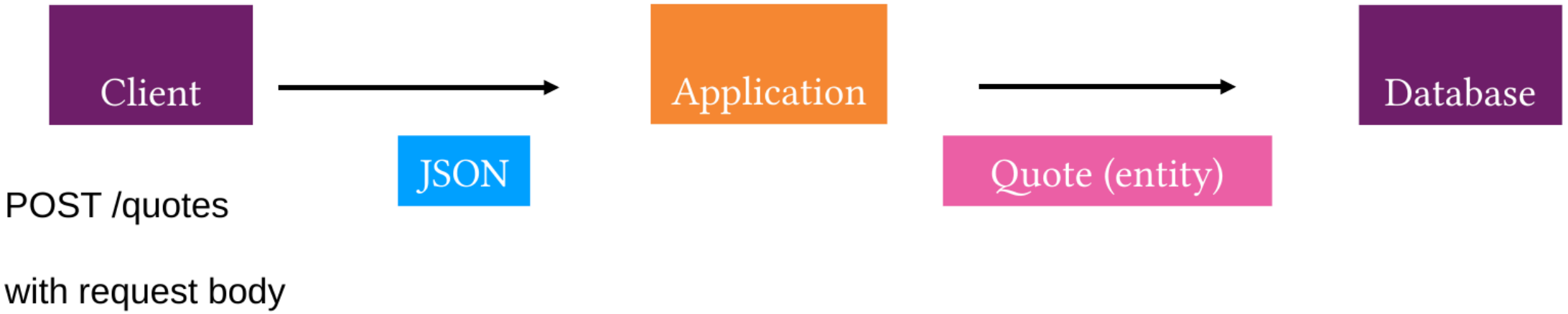
LECTURE

DTOs: Requests, Entities and Responses



© 2026 Inspire | Confidential

A JSON contract and an entity have different jobs



> A DTO describes the data crossing an application boundary.

© 2026 Inspire | Confidential

Why use a Data Transfer Object?

Separate concerns

Let the HTTP contract and persistence model evolve separately.

Choose exposed fields

Expose only fields the API promises. Keep internal fields internal.

Control the format

Adapt names and shapes without changing how data is stored.

Validate incoming data

A request DTO can carry input constraints. Mapping alone does not validate.

For this quote API, the response still includes amount_cents. A DTO is a design choice, not an automatic security guarantee.

© 2026 Inspire | Confidential

POST: the client supplies parcel inputs



POST /quotes

with request body

The server generates id and calculates amount_cents. QuoteRequest carries HTTP inputs; toInput() creates the ParcelInput used by the Service.

© 2026 Inspire | Confidential

GET: return the promised quote fields



The proposed response DTO preserves all five quote fields, including amount_cents.

© 2026 Inspire | Confidential

The request and response have different shapes

POST /quotes · request

```
{
  "weight_kg": 3,
  "zone": "local",
  "service": "standard"
}
```

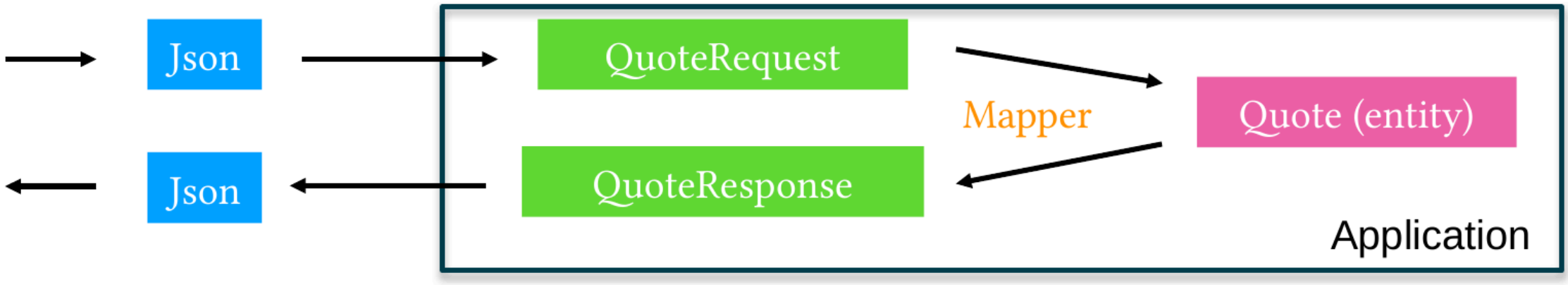
201 Created · response

```
{
  "id": 17,
  "weight_kg": 3,
  "zone": "local",
  "service": "standard",
  "amount_cents": 1300
}
```

> Three client inputs → five response fields. An illustrative generated id is shown.

© 2026 Inspire | Confidential

Make the field mapping explicit



Mapper is a conversion responsibility; a small method is enough. Pricing stays in the Service/Calculator. The diagram is not a full call stack.

© 2026 Inspire | Confidential

Response mapping: one small method can be enough

```
public static QuoteResponse from(Quote quote) {
  return new QuoteResponse(
      quote.getId(), quote.getWeight_kg(), quote.getZone(),
      quote.getService(), quote.getAmount_cents());
}
```

Keep the contract

Convert POST, GET, list and PUT responses.

Keep the tests

Run the existing API and storage checks.

Preview for the following optional Demo. No separate Mapper class or library is required.

© 2026 Inspire | Confidential

Request mapping: introduce QuoteRequest

```
public record QuoteRequest(
  int weight_kg, String zone, String service) {
  public ParcelInput toInput() {
      return new ParcelInput(weight_kg, zone, service);
  }
}
```

> id and amount_cents are server-owned outputs, not request inputs.

Preview for the optional refactoring. Keep existing shape checks and error rules.

Controller: JSON → QuoteRequest → ParcelInput. The Service calculates the amount; the request supplies no price.

© 2026 Inspire | Confidential

Apply the same questions to quote templates

Request

Template inputs

name · weight_kg
zone · service

Persistence

QuoteTemplate

Generated id
and template inputs

Response

Template result

id · name · weight_kg
zone · service

> Ask: which fields come from the client, which are stored, and which are returned?

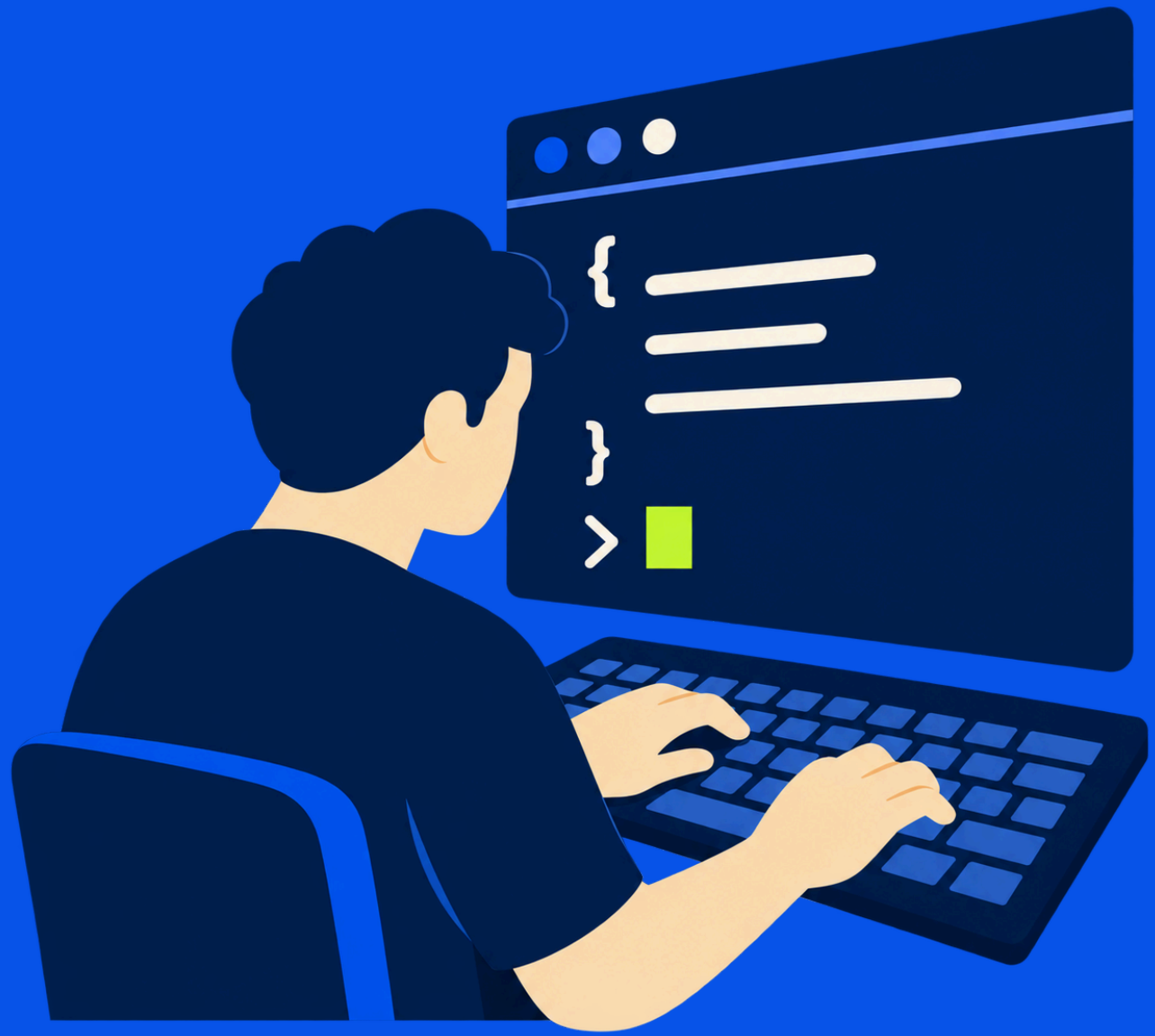
The Homework remains template persistence. Separate template DTOs or Mappers are not additional required work.

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Optional: Separate API Requests and Responses



© 2026 Inspire | Confidential